

RV1126B 加速运行目标检测

说明文档 v1.0

培 养 单 位 西北农林科技大学信息工程学院

合 作 单 位 武汉酷锆科技有限公司

文 档 作 者 许泽东

实 现 代 码 Python

指 导 教 师 张建锋

2026 年 03 月

1. 文档概述

本文档详细说明瑞芯微 RV1126B-P 开发板上加速运行目标检测（以 YOLOv5 为例）的完整流程，包括开发环境搭建、模型转换、代码优化、部署运行及问题排查，旨在帮助开发人员快速实现 RV1126B 平台上目标检测的高效运行，提升整体推理速度至 50FPS 以上。

本文档适用于基于 RV1126B-P 开发板（荣品电子 RP-RV1126B-P）、以 YOLOv5 为目标检测模型、采用 RKNN 加速方案的开发场景，适用于具备 Linux 基础、C/C++ 及 Python 开发经验的技术人员。

本文档的核心目标是：

- 完成红外图像目标检测模型 YOLOv5 的训练；
- 完成 YOLOv5 模型从 PyTorch 格式到 RKNN 格式的转换，适配 RV1126B 的 NPU 加速；
- 通过代码优化实现目标检测全流程加速；
- 完成模型部署与测试，确保目标检测功能正常且运行高效。

项目环境说明：

环境类型	具体配置	用途
开发板	荣品 RP-RV1126B-P（RV1126B-P 芯片，2+8 主板，12V 电源）	目标检测模型部署与运行
编译环境	Ubuntu20.04（AutoDL 服务器，Python3.8）	模型训练、转换及 C 语言项目交叉编译
交叉编译工具	gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu	编译适配 RV1126B（aarch64 架构）的 C 语言 Demo
核心工具	RKNN-Toolkit2 v2.3.2、rknn_model_zoo、YOLOv5	模型转换、Demo 开发、目标检测模型实现

2. 开发板准备与环境搭建

2.1 开发板硬件准备

- 硬件清单：RP-RV1126B-P 开发板（核心板+底板）、12V 电源、USB 转串口线、网线（可选）；
- 硬件连接：将 USB 转串口线连接开发板与电脑，接入 12V 电源，确保开发板正常供电。

2.2 串口连接与系统初始化

- 下载并安装 CP210x USB to UART Bridge 驱动（Silicon Labs 官网），安装完成后在电脑设备管理器中查看串口（通常为 COM3）。

2.3 串口调试（Mobaxterm）

- 打开 Mobaxterm，选择“Serial”连接，端口选择 COM3，波特率设置为 115200，其他参数默认；
- 连接后按回车键，或按下开发板上的 reset 键，即可看到系统启动日志（开发板已提前烧录系统，无需重新烧录）。

2.4 WIFI 连接与 SSH 配置

为方便后续部署，配置开发板连接 WIFI 并通过 SSH 远程控制：

- 启用 WIFI 接口：`ifconfig wlan0 up`；
- 扫描可用 WIFI：`iwlist wlan0 scanning`；
- 编辑 WIFI 配置文件：`vi /etc/wpa_supplicant.conf`，添加以下内容（替换为实际 WIFI 账号密码）：

```
ctrl_interface=/var/run/wpa_supplicant
ap_scan=1
update_config=1
network={
    ssid="MIX4" # WIFI 名称
    psk="ddddddddd" # WIFI 密码
    key_mgmt=WPA-PSK
}
```

- 连接 WIFI：`wpa_supplicant -Dnl80211 -c /etc/wpa_supplicant.conf -i wlan0 -B`；
- 获取 IP 地址：`udhcpc -i wlan0`，通过 `ifconfig wlan0` 查看 IP；
- SSH 连接：Mobaxterm 选择“SSH”，输入开发板 IP，用户名 root，密码 rpdzkj，连接成功后可拔掉串口线，仅保留电源连接。

3. 编译环境搭建（AutoDL 服务器）

3.1 基础环境安装

在 AutoDL 服务器（Ubuntu20.04）中安装 YOLOv5 所需依赖：

```
# 克隆 YOLOv5 仓库
git clone https://github.com/ultralytics/yolov5
cd yolov5
```

```
# 安装依赖（使用清华源加速）
```

```
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

3.2 RKNN-Toolkit2 安装

安装 RKNN-Toolkit2 用于模型转换，采用虚拟环境隔离，避免版本冲突：

```
# 创建并进入项目目录
```

```
mkdir project-Toolkit2 && cd project-Toolkit2
```

```
# 创建虚拟环境
```

```
python3 -m venv toolkit2_1.5
```

```
# 激活虚拟环境
```

```
source toolkit2_1.5/bin/activate
```

```
# 克隆 RKNN-Toolkit2 仓库
```

```
git clone https://github.com/airockchip/rknn-toolkit2.git
```

```
# 配置 pip 源
```

```
pip3 config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple/
```

```
# 安装依赖
```

```
cd rknn-toolkit2/rknn-toolkit2/packages/x86_64
```

```
pip install -r requirements_cp38-2.3.2.txt
```

```
# 安装 RKNN-Toolkit2（适配 Python3.8、x86_64 架构）
```

```
pip3 install rknn_toolkit2-2.3.2-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
```

```
# 验证安装（无报错即成功）
```

```
python3 -c "from rknn.api import RKNN;
```

3.3 交叉编译工具安装

下载并配置适配 RV1126B（aarch64 架构）的交叉编译工具：

```
# 下载交叉编译工具
```

```
wget
```

```
https://releases.linaro.org/components/toolchain/binaries/6.3-2017.05/aarch64-linux-gnu/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu.tar.xz
```

```
# 解压工具
```

```
tar xf gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-gnu.tar.xz
```

3.4 rknn_model_zoo 克隆

克隆瑞芯微官方 Demo 仓库，用于后续 C 语言项目编译：

```
git clone https://github.com/airockchip/rknn\_model\_zoo.git
```

4. 目标检测模型适配与转换

4.1 模型选择与训练（可选）

本文档以 YOLOv5 为例，可使用官方预训练模型，也可基于 FLIR_ADAS_v2 等数据集训练自定义模型（仅检测 person 和 car 类别），训练步骤如下（可选）：

➤ 下载 FLIR_ADAS_v2 数据集，筛选 person 和 car 类别，制作 YOLO 格式数据集；

➤ 在 yolov5/data 目录下创建 FLIRv2.yaml，配置数据集路径和类别：

```
train: ./FLIR_ADAS_v2/images/train
val: ./FLIR_ADAS_v2/images/val
test: ./FLIR_ADAS_v2/images/test
# Classes
names:
0: person
1: car
```

➤ 使用 screen 创建后台窗口，执行训练命令：

```
screen -U -S thermal_yolov5
```

➤ # 训练命令（使用 yolov5n.pt 轻量化模型，适配嵌入式平台）

```
python train.py --data FLIRv2.yaml --epochs 800 --weights 'yolov5n.pt' --cfg
yolov5n.yaml --batch-size 64 --imgsz 640 --rect --workers 16
```

➤ 训练完成后，在 runs/train/exp/weights 目录下获取 best.pt 模型。

4.2 PyTorch 模型转 ONNX 模型

由于 RKNN-Toolkit2 不直接支持 PyTorch 模型，需先将 .pt 模型转换为 ONNX 模型，转换前需修改 YOLOv5 源码适配转换要求：

➤ 修改 yolov5/models/yolo.py 的 forward 方法，简化输出：

```
def forward(self, x):
    z = [] # inference output
    for i in range(self.nl):
        x[i] = self.m[i](x[i]) # conv
    return x
```

➤ 修改 yolov5/export.py 的 run 方法，修正输出形状计算：

```
# 原代码
# shape = tuple((y[0] if isinstance(y, tuple) else y).shape)
# 修改后
```

```
shape = tuple(y[0].shape)
```

- 执行导出命令（以自定义模型为例，图像尺寸 512×640）：

```
python export.py --weights runs/train/exp/weights/best.pt --include onnx  
--imgsz 512 640 --simplify
```

- 导出完成后，在 weights 目录下获取 best.onnx 模型，同时将修改的 yolo.py 和 export.py 还原（避免影响后续使用）。

4.3 ONNX 模型转 RKNN 模型

将 ONNX 模型转换为适配 RV1126B 的 RKNN 模型，步骤如下：

- 将 best.onnx 复制到 RKNN-Toolkit2 的 YOLOv5 示例目录：

```
cp /root/autodl-tmp/yolov5/runs/train/exp/weights/best.onnx  
/root/autodl-tmp/project-Toolkit2/rknn-toolkit2/rknn-toolkit2/examples/onnx/yolov5
```

- 复制 test.py 为 test_modify.py，修改适配自定义模型：

```
修改目标平台为 RV1126B：rknn.config(mean_values=[[0, 0, 0]],  
std_values=[[255, 255, 255]], target_platform='rv1126b');
```

```
修改类别为 2 类：CLASSES = ("person", "car");
```

```
修正图像尺寸（匹配 ONNX 模型尺寸）：IMG_HEIGHT = 512,  
IMG_WIDTH = 640;
```

补全 sigmoid 激活（YOLOv5 输出层无激活，需手动添加），修改 process 函数中相关代码。

- 执行 test_modify.py，导出 RKNN 模型。
- 导出成功后，获取 best.rknn 模型，将其复制到 rknn_model_zoo 的示例模型目录：

```
cp best.rknn /root/autodl-tmp/rknn_model_zoo/examples/yolov5/model/
```

5. 代码优化与交叉编译

5.1 核心优化：解决 RGA 报错与预处理加速

RV1126B 运行时会出现 RGA 错误（核心原因：RGA 驱动版本与库版本不匹配），虽不影响功能，但会导致预处理耗时增加。由于使用固定尺寸图像，可取消 RGA 预处理，直接拷贝原始图像数据，提升速度，修改 yolov5/cpp/rknpu2/yolov5.cc 的 inference_yolov5_model 函数：

```
// 注释原有 letterbox 预处理代码  
// ret = convert_image_with_letterbox(img, &dst_img, &letter_box, bg_color);  
// 手动设置 letterbox 参数（无缩放、无填充，与原函数“无变化”效果一致）  
letter_box.scale = 1.0f; // 缩放比例=1（无缩放）
```

```
letter_box.x_pad = 0;    // 水平填充=0
letter_box.y_pad = 0;    // 垂直填充=0
memcpy(dst_img.virt_addr, img->virt_addr, dst_img.size); //拷贝原始图像数据
```

5.2 新增时间统计：监控各阶段耗时

在 `inference_yolov5_model` 函数中添加时间统计，监控预处理、推理、后处理等各阶段耗时，便于优化分析，修改内容如下：

- 新增时间统计头文件：`#include <time.h>`;
- 定义时间统计变量（总耗时、各阶段耗时）;
- 在各阶段开始和结束时记录时间;
- 计算各阶段耗时（转换为毫秒），并打印统计结果。

5.3 批量推理优化：支持多图像循环推理

修改 `yolov5/cpp/main.cc`，实现多图像循环推理，统计平均耗时，适配批量检测场景，核心修改如下：

- 新增高精度计时头文件：`#include <sys/time.h>`;
- 定义循环推理次数：`#define LOOP_COUNT 100`;
- 添加计时变量，存储每次推理耗时;
- 循环执行推理，计算并打印每次耗时、总耗时及平均耗时。

5.4 C 语言项目交叉编译

基于 `rknn_model_zoo` 编译适配 RV1126B 的 C 语言 Demo，步骤如下：

- 复制 YOLOv5 示例项目，修改适配自定义模型：

```
cp -r /root/autodl-tmp/rknn_model_zoo/examples/yolov5/
/root/autodl-tmp/rknn_model_zoo/examples/yolov5_modify
```

- 修改 `postprocess.h`，将类别数量改为 2：

```
// #define OBJ_CLASS_NUM 80
#define OBJ_CLASS_NUM 2
```

- 修改 `model` 目录下的 `coco_80_labels_list.txt`，仅保留 `person` 和 `car` 两个类别;
- 执行交叉编译命令（指定交叉编译工具路径）：

```
cd /root/autodl-tmp/rknn_model_zoo
# 导出交叉编译工具路径（替换为实际解压路径）
export
GCC_COMPILER=/root/autodl-tmp/gcc-linaro-6.3.1-2017.05-x86_64_aarch64-linux-
gnu/bin/aarch64-linux-gnu
# 赋予脚本执行权限
```

```
chmod +x build-linux.sh
# 编译 yolov5_modify 项目（适配 RV1126B、aarch64 架构）
./build-linux.sh -t rv1126b -a aarch64 -d yolov5_modify
```

- 编译完成后，在 `rknn_model_zoo/install` 目录下获取编译好的项目（`rv1126b_linux_aarch64/rknn_yolov5_modify_demo`）。

6. 模型部署与运行测试

6.1 项目上传至开发板

- 通过 SSH 将编译好的 `rknn_yolov5_modify_demo` 文件夹上传至 RV1126B 开发板（示例路径：`/root/raw_c/`）：

```
# 在 AutoDL 服务器中执行（替换为开发板 IP）
scp -r /root/autodl-tmp/rknn_model_zoo/install/rv1126b_linux_aarch64/rknn_yolov5_modify_demo root@192.168.1.100:/root/raw_c/
```

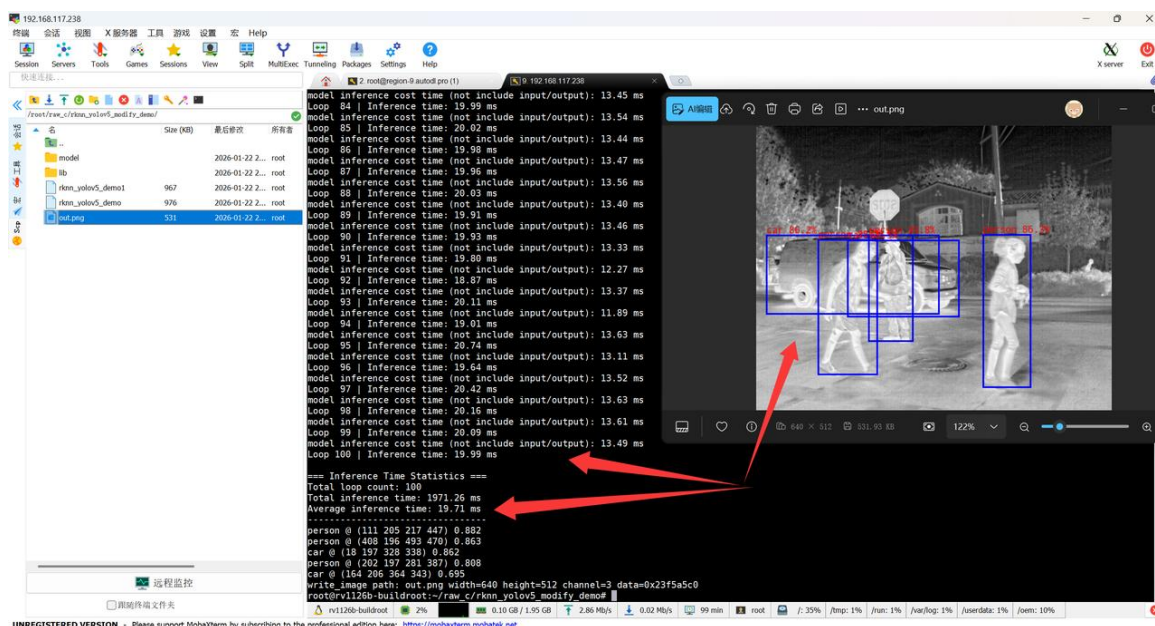
6.2 运行目标检测程序

在开发板中进入项目目录，执行运行命令，测试目标检测功能：

```
# 进入项目目录
cd /root/raw_c/rknn_yolov5_modify_demo
# 赋予程序执行权限
chmod +x rknn_yolov5_demo
# 运行程序（指定 RKNN 模型路径和测试图像路径）
./rknn_yolov5_demo ./model/best.rknn ./model/infrared.jpg
```

6.3 运行结果验证

- 运行成功后，会打印各阶段耗时统计（预处理、推理、后处理等），以及目标检测结果（类别、坐标、置信度）；
- 程序会生成 `out.png` 文件，包含检测框和类别标注，可通过 SSH 下载至电脑查看；
- 循环推理测试时，会打印每次推理耗时、总耗时及平均耗时，验证加速效果（取消 RGA 预处理后，预处理耗时显著降低）。



图表 1 RV1126B 目标检测运行结果